elixir_examples/lib/examples/structs/sms_2.ex
```
defmodule Examples.Structs.Sms do
  @enforce_keys [:from, :to]
  defstruct @enforce_keys ++ [:body, status: "delivered"]

  def other_party(sms) do
    case sms.status do
      "delivered" -> sms.to
      "received" -> sms.from
    end
  end
end
```

```
$ iex -S mix
iex> sms = %Examples.Structs.Sms{}
** (ArgumentError) the following keys must also be given when
    building struct Examples.Structs.Sms: [:from, :to]

iex> sms = %Examples.Structs.Sms{to: "you", from: "me"}
%Examples.Structs.Sms{body: nil, from: "me", status: "delivered", to: "you"}

iex> Examples.Structs.Sms.other_party(sms)
"you"
```

There's nothing stopping you from initializing this with bad data, but required keys (via the @enforce_keys module attribute) and default values significantly improve the developer experience of your application data structures.

The function other_party/1 demonstrates a common technique of putting functions that operate on a struct in the same file as the struct. Let's see how this is used to implement contained data structures.

Implementing Data Structures

Data is the heart of programming. You need to be able to create clean data structures so that you can hold, read, and modify data easily. Doing this well will make it easier to read and maintain software applications.

Ruby is object-oriented. So, you write a class that has instance variables and public or private methods. You modify the data through these methods and build your program by implementing your application domain through classes. At runtime, classes are initialized as objects that hold a specific set of data.

But Elixir is functional. It doesn't have classes, and data is immutable, so you might be scratching your head at how to go about implementing your data structures. We can take what we've learned so far—modules, functions, and structs—to implement a clean data layer.